

WireGuard VPN Deployment Guide for Distribution Management System

Last updated: May 7, 2026

This guide explains how to deploy this project on a data center server and restrict access using WireGuard VPN. It includes project-specific configuration changes and device connection steps. It does not create any additional files.

1) High-Level Architecture

- Public Internet: only UDP 51820 exposed for WireGuard.
- Private VPN: clients get 10.0.0.0/24 addresses.
- App access: only via VPN (HTTPS) to the server.
- Docker services: backend, frontend, and MySQL are private, accessed via an internal Docker network and an HTTPS reverse proxy.

Traffic flow:

1. Client connects to WireGuard server over UDP 51820.
2. Client receives VPN IP (e.g., 10.0.0.2).
3. Client opens `https://10.0.0.1` (VPN gateway) to reach the app.
4. Reverse proxy forwards to backend and frontend containers.

2) Server Requirements

- Ubuntu 22.04+ (or comparable Linux).
- Static public IP.
- Docker + Docker Compose.
- WireGuard.
- Reverse proxy with TLS (Nginx or Caddy).

Open ports:

- UDP 51820: WireGuard
- TCP 443: HTTPS (restricted to VPN subnet)
- TCP 22: SSH admin access (restrict by IP if possible)

3) WireGuard Server Setup (Generic)

Commands below are examples. Adjust for your OS.

Install WireGuard:

- Ubuntu: `sudo apt install wireguard wireguard-tools`

Generate server keys:

- `wg genkey | tee /etc/wireguard/server_privatekey | wg pubkey > /etc/wireguard/server_publickey`
- `wg genpsk > /etc/wireguard/server_presharedkey` (optional but recommended)

Server config example (wg0.conf):

```
[Interface]
PrivateKey = <SERVER_PRIVATE_KEY>
Address = 10.0.0.1/24
ListenPort = 51820

PostUp = iptables -A FORWARD -i wg0 -j ACCEPT; iptables -A FORWARD -o wg0 -j
ACCEPT; iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
PostDown = iptables -D FORWARD -i wg0 -j ACCEPT; iptables -D FORWARD -o wg0 -j
ACCEPT; iptables -t nat -D POSTROUTING -o eth0 -j MASQUERADE

# Add peers below
[Peer]
PublicKey = <CLIENT1_PUBLIC_KEY>
PresharedKey = <SERVER_PRESHARED_KEY>
AllowedIPs = 10.0.0.2/32
```

Enable IP forwarding:

- `sudo sysctl -w net.ipv4.ip_forward=1`
- Persist in /etc/sysctl.conf: `net.ipv4.ip_forward = 1`

Start WireGuard:

- `sudo systemctl enable wg-quick@wg0`
- `sudo systemctl start wg-quick@wg0`
- `sudo wg show`

4) Reverse Proxy and HTTPS

Only allow HTTPS access from the VPN subnet. Use a self-signed certificate or internal CA.

Recommended:

- Terminate TLS at Nginx/Caddy.
- Proxy /api to backend, / to frontend.
- Add security headers and rate limiting.

5) Project-Specific Configuration Changes

This section maps directly to the current project structure.

5.1 Backend environment variables

File: backend/.env (do not commit secrets)

Set production values:

- `DEBUG=false`
- `ENVIRONMENT=production`
- `ENFORCE_HTTPS=true`
- `CSRF_COOKIE_SECURE=true`

Set strong secrets:

- `SECRET_KEY`: generate with `python -c "import secrets; print(secrets.token_urlsafe(64))"`
- `DB_PASSWORD`: strong random password
- `ADMIN_INITIAL_PASSWORD`: strong random password

Update CORS to only allow VPN access and proxy:

- `CORS_ORIGINS=https://10.0.0.1,http://172.20.0.5`

5.2 Backend config defaults

File: `backend/app/config.py`

Keep defaults for dev, but production should rely on `.env` overrides. No code change required if `.env` is set.

5.3 Frontend API base URL

File: `frontend/src/services/api.js`

Already uses:

- `import.meta.env.VITE_API_URL || 'http://localhost:8080/api'`

In production, build with:

- `VITE_API_URL=https://10.0.0.1/api`

5.4 Docker Compose strategy

Goal: expose only HTTPS (443) and WireGuard (51820). Do not expose backend or database to the public.

Production pattern:

- MySQL: no host port binding
- Backend: no host port binding
- Frontend: no host port binding
- Reverse proxy: host ports 443 and 80 (80 for redirect)

If you keep using the current `docker-compose.yml`, remove or replace:

- Backend `ports: "8080:8080"`
- Frontend `ports: "5173:5173"`

Instead, use `expose` for internal access and a proxy container for external access.

5.5 HTTPS enforcement

Backend already includes HTTPS enforcement middleware in backend/app/main.py via ENFORCE_HTTPS. Set `ENFORCE_HTTPS=true` in production.

6) Suggested Docker Deployment Topology

- Docker network: 172.20.0.0/16
- MySQL: 172.20.0.2
- Backend: 172.20.0.3
- Frontend: 172.20.0.4
- Reverse proxy: 172.20.0.5

Only the reverse proxy publishes port 443 to host.

7) Device Connection (Client Setup)

For each user/device:

1. Generate client keys on admin machine:

- `wg genkey | tee client_privatekey | wg pubkey > client_publickey`

2. Add peer to server wg0.conf:

```
[Peer]
PublicKey = <CLIENT_PUBLIC_KEY>
PresharedKey = <SERVER_PRESHARED_KEY>
AllowedIPs = 10.0.0.2/32
```

3. Create client config (client.conf):

```
[Interface]
PrivateKey = <CLIENT_PRIVATE_KEY>
Address = 10.0.0.2/32
DNS = 8.8.8.8

[Peer]
PublicKey = <SERVER_PUBLIC_KEY>
PresharedKey = <SERVER_PRESHARED_KEY>
Endpoint = <SERVER_PUBLIC_IP>:51820
AllowedIPs = 10.0.0.0/24
PersistentKeepalive = 25
```

4. Import into WireGuard client app (Windows, macOS, Linux, iOS, Android).

5. Validate:

- `ping 10.0.0.1`
- `curl -k https://10.0.0.1/api/health`

8) Access Control Policy

- Assign one VPN IP per user/device.
- Remove peers from wg0.conf to revoke access.
- Use short token lifetimes in backend:
 - ACCESS_TOKEN_EXPIRE_MINUTES=30

9) Firewall Rules (Recommended)

- Allow UDP 51820 from anywhere (for WireGuard handshake).
- Allow TCP 443 only from 10.0.0.0/24.
- Allow TCP 22 only from admin IP range.
- Deny everything else.

10) Operations and Monitoring

- WireGuard status: `sudo wg show`
- Docker status: `docker ps`
- Backend logs: `docker logs dms-backend`
- Database health: `docker exec dms-mysql mysqladmin ping`

11) Troubleshooting

- Client cannot connect: check UDP 51820, server public IP, and peer keys.
- Connected but no access: check firewall rules and reverse proxy.
- CORS errors: update CORS_ORIGINS in backend/.env.
- HTTPS redirect loops: confirm reverse proxy sends X-Forwarded-Proto.

12) Final Deployment Checklist

- WireGuard running and peers added.
- Reverse proxy active with HTTPS cert.
- Backend .env in production mode.
- Docker services running with internal-only ports.
- Client can reach https://10.0.0.1 via VPN.
- Admin credentials rotated.

If you want me to tailor this guide to a specific OS (Ubuntu, RHEL, Windows Server) or a specific reverse proxy (Nginx vs Caddy), tell me which one you are using.